Visualização Geométrica, Rasterização e Recorte de Árvores Arteriais Geradas pelo Método CCO

Heitor Augusto de Oliveira Ramos[1], **Rafael Stylita Duarte**[1]

[1]Departamento de Computação (DECOM) — Universidade Federal de Ouro Preto (UFOP)

Trabalho Prático 1 — BCC327 Computação Gráfica — 2026.2 — Prof. Dr. Rafael Alves Bonfim de Queiroz

RESUMO

Contexto: Árvores arteriais sintéticas geradas pelo método *Constrained Constructive Optimization* (CCO) são distribuídas em arquivos VTK que guardam apenas pontos, conectividade e o raio de cada vaso; transformar esses dados em imagem exige todas as etapas clássicas do *pipeline* gráfico. **Objetivo:** Desenvolver uma aplicação única que leia, represente, transforme, rasterize e recorte os modelos CCO 2D e 3D fornecidos, deixando explícito o que é implementação própria e o que vem da biblioteca. **Metodologia:** Em Python, implementamos um leitor de VTK ASCII, matrizes homogêneas 4×4, Bresenham, *Scanline Fill* com tabelas de arestas, supersampling por cobertura, Cohen–Sutherland, Liang–Barsky e Sutherland–Hodgman. A renderização interativa (tubos, câmera, projeções, MSAA/SSAA) usa PyVista/VTK. **Resultados:** Os dois modelos (128 pontos, 127 segmentos cada) são exibidos como tubos com cor e espessura proporcionais ao raio. O supersampling 4×4 reduz em cerca de 11 vezes o erro de cobertura de uma reta inclinada em relação à amostragem simples. Cohen–Sutherland e Liang–Barsky produzem recortes idênticos nos 127 segmentos; o primeiro resolve 114 deles trivialmente e calcula apenas 14 interseções. **Conclusão:** Os sete módulos do enunciado foram atendidos; o caso côncavo do Sutherland–Hodgman evidencia quando o Weiler–Atherton é necessário.

Palavras-Chave: Computação Gráfica; Método CCO; Rasterização; Antialiasing; Recorte; VTK.

1 Introdução

A simulação de escoamento sanguíneo e o estudo da perfusão de tecidos dependem de representações geométricas da árvore vascular. Como imagens médicas não resolvem os vasos mais finos, modelos sintéticos são uma alternativa. O método CCO, introduzido por Schreiner e Buxbaum [1], constrói essas árvores adicionando um segmento terminal por vez e otimizando a posição de cada nova bifurcação sob restrições fisiológicas.

Uma árvore CCO, porém, é apenas um conjunto de números: coordenadas de pontos, pares de índices e um raio por segmento. Para que ela se torne uma imagem, é preciso escolher primitivas, mapear atributos em cor, posicionar o observador, projetar, converter a geometria contínua em pixels e descartar o que está fora da área de visualização. Este trabalho percorre essas etapas com os modelos `tree2D_Nterm064.vtk` e `tree3D_Nterm064.vtk` fornecidos pelo professor.

Objetivos. Construir uma aplicação que (i) leia e interprete os modelos VTK; (ii) represente a árvore por tubos com cor e espessura em função do raio; (iii) aplique translação, rotação e escala e ofereça diferentes vistas e projeções; (iv) implemente Bresenham e *Scanline Fill*; (v) compare imagens com e sem antialiasing; (vi) implemente e compare Cohen–Sutherland e Liang–Barsky; e (vii) implemente Sutherland–Hodgman e discuta o Weiler–Atherton. A Seção 2 resume a teoria, a Seção 3 descreve a arquitetura, a Seção 4 apresenta os resultados e a Seção 5 discute limitações e escolhas.

2 Fundamentação Teórica

2.1 Árvores CCO e o formato VTK

No CCO, a árvore cresce dentro de um domínio de perfusão: cada novo terminal é conectado ao segmento que minimiza uma função de custo (tipicamente o volume total), e os raios são reajustados

para manter pressões e fluxos consistentes [1]. Neste trabalho não regeramos as árvores: usamos os modelos prontos, cada um com $N_{term} = 64$ terminais, 128 pontos e 127 segmentos. Os arquivos estão no formato VTK legado ASCII, `DATASET POLYDATA`: a seção `POINTS` lista as coordenadas (x, y, z), a seção `LINES` lista cada segmento como `2 id_início id_fim`, e `CELL_DATA` traz o escalar `raio`, um valor por segmento [10].

2.2 Transformações geométricas e observador

Em coordenadas homogêneas, o ponto $\mathbf{p} = (x, y, z, 1)^T$ é transformado por matrizes 4×4, e translações passam a ser produtos de matrizes como as demais transformações [2, 3]:

$$T(\mathbf{d}) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad S(s) = \begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{1}$$

Para girar ou escalar em torno de um pivô $\mathbf{c}$ (o centroide da árvore), usa-se $M' = T(\mathbf{c})\, M\, T(-\mathbf{c})$, e a matriz acumulada é $M_{acc} \leftarrow M' M_{acc}$.

Espaço do mundo × sistema da câmera. As coordenadas do VTK, depois das transformações de modelagem, estão no *espaço do mundo*, comum a todos os objetos da cena. A câmera é definida por posição $\mathbf{e}$, ponto focal $\mathbf{f}$ e vetor *up*. Com eles se monta uma base ortonormal $(\mathbf{u}, \mathbf{v}, \mathbf{w})$, com $\mathbf{w} = (\mathbf{e} - \mathbf{f})/\|\mathbf{e} - \mathbf{f}\|$, e a matriz de visualização leva cada ponto do mundo para o *sistema da câmera*, onde o observador está na origem olhando ao longo de $-\mathbf{w}$ [4]. Transformar o modelo muda as coordenadas dos pontos no mundo; mover a câmera muda apenas essa mudança de base. Na aplicação as duas operações são separadas: as teclas de transformação alteram os pontos, e as teclas de vista alteram a câmera.

Projeções. Na projeção ortográfica os projetores são paralelos e tamanhos não dependem da profundidade ($x_p = x_c$, $y_p = y_c$). Na perspectiva, os projetores convergem no centro de projeção e $x_p = d\, x_c/(-z_c)$, $y_p = d\, y_c/(-z_c)$: objetos mais distantes aparecem menores.

2.3 Rasterização

Bresenham. Para uma reta no primeiro octante ($0 \le m \le 1$), em cada coluna x escolhe-se entre o pixel $(x + 1, y)$ e o $(x + 1, y + 1)$ pelo sinal de uma variável de decisão atualizada só com somas inteiras [5]. Usamos a forma generalizada para os oito octantes, com erro $e = \Delta x - \Delta y$ e $e_2 = 2e$: se $e_2 > -\Delta y$ avança-se em x, e se $e_2 < \Delta x$ avança-se em y. São gerados $\max(|\Delta x|, |\Delta y|) + 1$ pixels com conectividade de 8 vizinhos. O DDA chega ao mesmo resultado, mas usa divisão e arredondamento em ponto flutuante a cada passo.

Scanline Fill. Cada aresta não horizontal entra na Tabela de Arestas (ET), indexada pela primeira linha inteira que cruza, com y_{fim} e o incremento $1/m = \Delta x/\Delta y$. Para cada linha y, a Tabela de Arestas Ativas (AET) recebe as arestas que começam, perde as que terminam, é ordenada por x, e os pixels entre pares consecutivos de interseções são preenchidos (regra par-ímpar). Depois, cada x é atualizado incrementalmente com $x \leftarrow x + 1/m$. O intervalo semiaberto $[y_{min}, y_{max})$ de cada aresta impede que um vértice compartilhado seja contado duas vezes.

2.4 Aliasing e antialiasing

Amostrar uma primitiva contínua com uma única amostra por pixel produz *aliasing*: bordas em degrau e vasos finos que somem ou se fragmentam quando sua largura é menor que um pixel. O *supersampling* (SSAA) avalia $k \times k$ amostras por pixel e usa a fração coberta como intensidade, aproximando a integral de cobertura do pixel. O MSAA, usado pelas GPUs, guarda várias amostras de cobertura e profundidade por pixel, mas avalia a cor uma única vez por primitiva; por isso suaviza as bordas geométricas com custo menor que o SSAA, mas não trata o aliasing interno das texturas e do sombreamento.

2.5 Recorte de segmentos

Cohen–Sutherland [2]. Cada extremo recebe um código de 4 bits (acima, abaixo, direita, esquerda da janela). Se $c_0 \vee c_1 = 0$, o segmento é aceito trivialmente; se $c_0 \wedge c_1 \neq 0$, os dois extremos estão do mesmo lado de fora e o segmento é rejeitado trivialmente. Caso contrário, um extremo de fora é movido para a interseção com uma borda e o teste se repete.

Liang–Barsky [6]. Com $P(u) = P_0 + u\,(P_1 - P_0)$, $0 \le u \le 1$, as quatro condições de pertinência são escritas como $p_k u \le q_k$, com $p = (-\Delta x, \Delta x, -\Delta y, \Delta y)$ e $q = (x_0 - x_{min},\ x_{max} - x_0,\ y_0 - y_{min},\ y_{max} - y_0)$. Se $p_k < 0$ o segmento entra pela borda k e $u_1 = \max(u_1, q_k/p_k)$; se $p_k > 0$ ele sai e $u_2 = \min(u_2, q_k/p_k)$; se $p_k = 0$ e $q_k < 0$ o segmento é paralelo à borda e está fora. O trecho visível é $[u_1, u_2]$, se $u_1 \le u_2$. Cada extremo é calculado uma única vez, sem as interseções intermediárias do Cohen–Sutherland.

Cyrus–Beck (discutido, não implementado) [7]. Usa a mesma ideia paramétrica, mas para uma janela *convexa qualquer*: para cada aresta i da janela com normal interna $\mathbf{N}_i$ e um ponto $\mathbf{E}_i$ sobre ela, resolve $\mathbf{N}_i \cdot (P(t) - \mathbf{E}_i) = 0$ e classifica t como entrada ou saída pelo sinal de $\mathbf{N}_i \cdot (P_1 - P_0)$. A principal diferença é a janela: Cohen–Sutherland e Liang–Barsky exigem um retângulo alinhado aos eixos (códigos de região e as quatro comparações fixas), enquanto Cyrus–Beck aceita qualquer polígono convexo, ao custo de produtos escalares por aresta. Liang–Barsky é exatamente o Cyrus–Beck com normais $(\pm 1, 0)$ e $(0, \pm 1)$.

2.6 Recorte de polígonos

Sutherland–Hodgman [8]. O polígono é recortado sucessivamente contra o semiplano interno de cada aresta da janela convexa. Para cada aresta (S, E) do polígono há quatro casos: dentro→dentro emite E; dentro→fora emite a interseção; fora→dentro emite a interseção e E; fora→fora não emite nada. A saída é sempre *uma única* lista de vértices.

Weiler–Atherton (discutido) [9]. Calcula as interseções entre as fronteiras do polígono e da janela, insere-as nas duas listas de vértices marcadas como entrada ou saída e percorre as listas alternando entre polígono e janela a cada interseção. Por isso pode devolver *várias* regiões separadas e aceita janelas côncavas e polígonos com buracos. É o algoritmo adequado quando o polígono é côncavo e o recorte o divide em mais de uma região (a saída do Sutherland–Hodgman ligaria as regiões por arestas sobrepostas ao longo da borda da janela), quando a própria janela é côncava, quando há buracos ou várias regiões, e em operações booleanas entre polígonos (união, interseção, diferença). O custo é uma implementação mais complexa, com casos especiais para vértices sobre arestas e interseções tangentes.

3 Metodologia e Arquitetura da Aplicação

A aplicação foi escrita em Python 3.12, com NumPy para a álgebra linear e PyVista [11] (sobre o VTK [10]) para a renderização interativa, e tem um único ponto de entrada, `main.py`, com três modos: `viewer` (visualizador interativo), `plots` (figuras dos Módulos 3 a 7) e `screenshots` (capturas *off-screen* do visualizador). A Figura 1 mostra o fluxo e a Tabela 1 relaciona os arquivos aos módulos do enunciado.

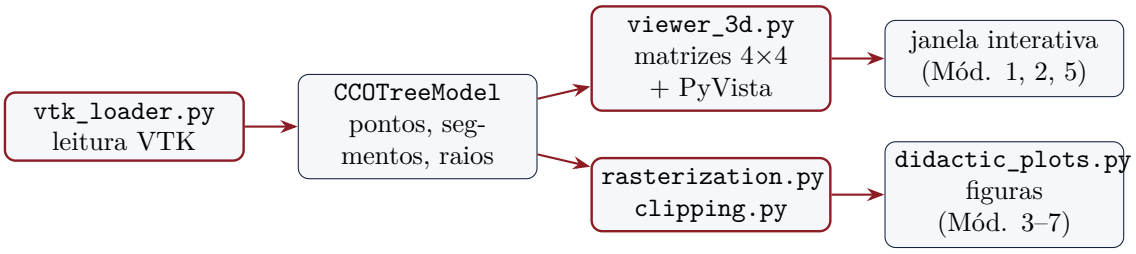


Figura 1: Arquitetura da aplicação. Bordas vermelhas: código com algoritmos implementados pelo grupo.

Tabela 1: Módulos do enunciado, arquivos e divisão entre código próprio e biblioteca.

Módulo	**Arquivo**	**Implementado pelo grupo**	**Biblioteca**
1 Leitura/primitivas	`vtk_loader.py`, `viewer_3d.py`	leitor VTK, um tubo por segmento	`vtkTubeFilter`, mapa de cores
2 Transformações	`viewer_3d.py`	matrizes T, $R_{x,y,z}$, S, pivô, acúmulo	câmera, projeções
3 Linhas	`rasterization.py`	Bresenham (8 octantes)	—
4 Polígonos	`rasterization.py`	Scanline com ET/AET	—
5 Antialiasing	`didactic_plots.py`	SSAA por cobertura e métricas	MSAA/SSAA no visualizador
6 Recorte de segmentos	`clipping.py`	Cohen–Sutherland, Liang–Barsky	—
7 Recorte de polígonos	`clipping.py`	Sutherland–Hodgman	—

Leitura (Módulo 1). O leitor não usa nenhuma função de leitura do VTK: lê o arquivo como uma sequência de *tokens* e interpreta as palavras-chave `POINTS`, `LINES`, `CELL_DATA/POINT_DATA` e `SCALARS`. Polilinhas com mais de dois pontos são expandidas em segmentos consecutivos que herdam o atributo da célula. O resultado é um `CCOTreeModel` com os arrays de pontos $(N, 3)$, segmentos $(M, 2)$ e raios (M). O modelo é classificado como 2D quando a variação de z é nula.

Primitivas e cor (Módulo 1). Cada segmento vira um tubo com os seus próprios dois pontos (os pontos das bifurcações são duplicados). Se os pontos fossem compartilhados, o filtro de tubos interpolaria o raio entre vasos vizinhos e um ramo fino herdaria parte da espessura do vaso pai. O raio do tubo é $r_{min}^{mundo} \cdot r/r_{min}$, ou seja, *proporcional* ao atributo; o fator `RADIUS_TO_WORLD` só converte a unidade do arquivo para a escala da cena. Para a cor usamos o mapa contínuo divergente `coolwarm` (azul = fino, vermelho = grosso). Como o raio é um escalar contínuo, um mapa contínuo representa melhor suas variações do que uma paleta RGB discreta, que criaria fronteiras artificiais entre classes. A tecla `C` deixa a espessura constante, e então só a cor codifica o raio.

Transformações e câmera (Módulo 2). As matrizes são montadas em NumPy e aplicadas aos pontos em coordenadas homogêneas; a malha é reconstruída a cada operação. O pivô é o centroide atual, de modo que uma operação seguida da inversa volta ao estado anterior. O terminal imprime a matriz acumulada a cada passo. As vistas frontal, lateral, superior e isométrica e as projeções ortográfica e perspectiva são definidas na câmera do PyVista, e a posição, o foco e o *up* da câmera são impressos a cada mudança.

Módulos didáticos (3 a 7). Os algoritmos de rasterização e recorte são funções Python puras, sem NumPy nem bibliotecas gráficas, que recebem coordenadas e devolvem listas de pixels ou de vértices. O Matplotlib só exibe essas listas. Os algoritmos de recorte de segmentos aceitam um objeto `ClipStats` que conta iterações, divisões e casos triviais. Para o antialiasing, a função `rasterize_capsules` rasteriza segmentos com espessura (cápsulas) avaliando a distância de cada subamostra ao segmento; com $k = 1$ há uma amostra no centro do pixel (sem antialiasing), e com $k > 1$ há $k \times k$ amostras. Os casos com e sem antialiasing usam a mesma geometria e a mesma espessura, e o erro é medido contra uma referência com 16×16 amostras.

4 Resultados

4.1 Módulo 1: leitura, primitivas e atributos

A Tabela 2 mostra as informações impressas ao carregar cada modelo. A Figura 2 mostra a árvore 3D com cor e espessura proporcionais ao raio, a mesma árvore com espessura constante e a árvore 2D.

Com a espessura proporcional, o tronco e os ramos principais se destacam; com a espessura constante, a hierarquia fica visível apenas pela cor.

Tabela 2: Resumo dos modelos carregados (saída do Módulo 1).

Modelo	Pontos	Segmentos	Raio mín.	Raio máx.	r_{max}/r_{min}	Dimensão
`tree2D_Nterm064`	128	127	0,2032137	1,1813672	5,81	2D ($z = 0$)
`tree3D_Nterm064`	128	127	0,2052275	1,0178490	4,96	3D

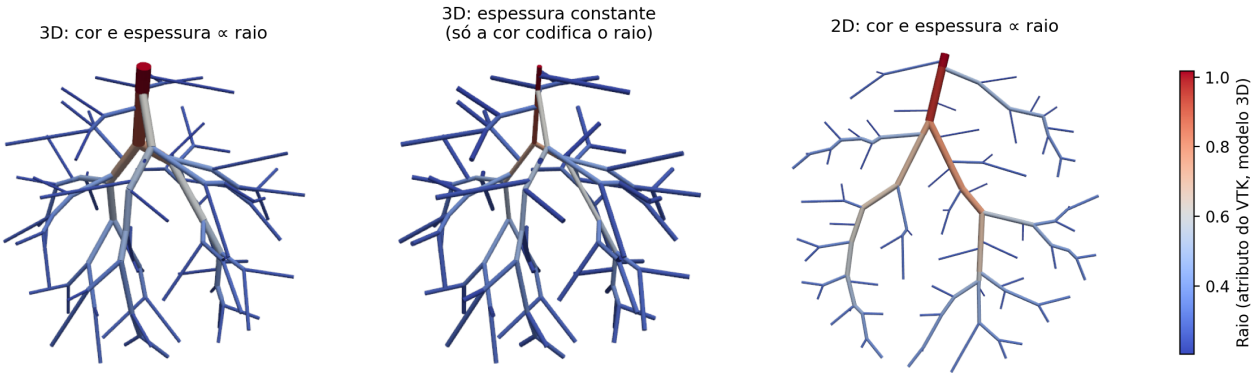


Figura 2: Módulo 1: tubos coloridos pelo raio (mapa `coolwarm`). Esquerda: modelo 3D com espessura proporcional ao raio; centro: espessura constante; direita: modelo 2D.

4.2 Módulo 2: transformações, vistas e projeções

A Figura 3 mostra, com a câmera fixa, a árvore original e o resultado de uma translação, de uma rotação de 45° e de uma escala uniforme de $1{,}1^5 \approx 1{,}61$. Como a câmera não se move, a mudança na imagem vem apenas da alteração das coordenadas no espaço do mundo. A Figura 4 mostra quatro posições do observador, e a Figura 5 compara as duas projeções com a mesma câmera. Na perspectiva, os ramos mais próximos do observador aparecem maiores e as retas paralelas convergem; na ortográfica, segmentos de mesmo comprimento aparecem iguais em qualquer profundidade, o que facilita comparar comprimentos e raios, mas tira a noção de profundidade.

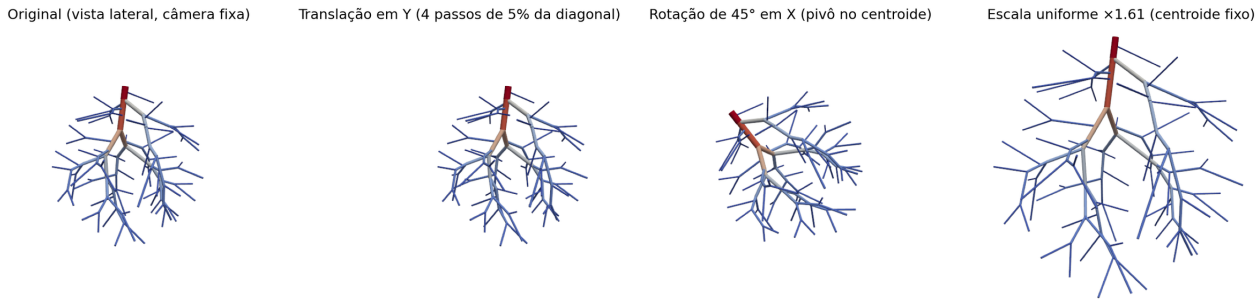


Figura 3: Módulo 2: transformações aplicadas pelas matrizes 4×4 implementadas pelo grupo, com a câmera fixa (vista lateral).

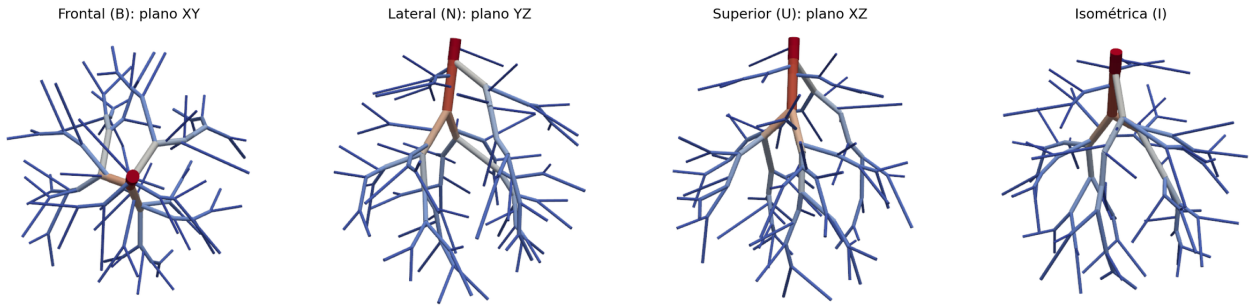


Figura 4: Módulo 2: quatro posições do observador (teclas `B`, `N`, `U` e `I`).

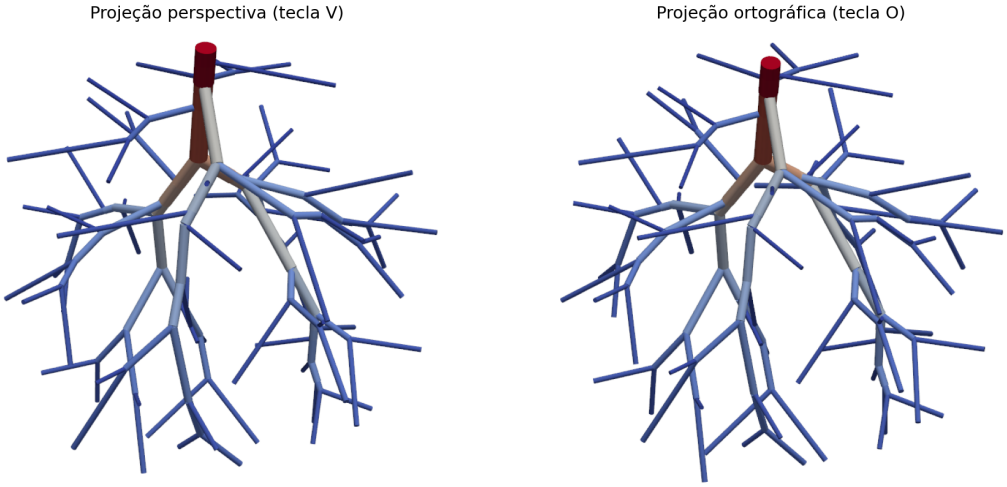


Figura 5: Módulo 2: projeção perspectiva × ortográfica com a mesma posição de câmera.

4.3 Módulo 3: rasterização de linhas

A Tabela 3 mostra a execução do Bresenham de $(2, 3)$ a $(14, 10)$: $\Delta x = 12$, $\Delta y = 7$ e erro inicial $e = 5$. São gerados $\max(12, 7) + 1 = 13$ pixels, e em cada coluna a distância vertical entre o centro do pixel escolhido e a reta ideal é de no máximo meio pixel (Figura 6, esquerda). À direita, os 127 segmentos da árvore 2D foram mapeados numa grade 60×60 com a mesma escala em x e y e rasterizados com a mesma função.

Tabela 3: Execução do Bresenham de $(2, 3)$ a $(14, 10)$. A cada passo: avança x se $e_2 > -7$; avança y se $e_2 < 12$.

Passo	0	1	2	3	4	5	6	7	8	9	10	11	12
Pixel	(2,3)	(3,4)	(4,4)	(5,5)	(6,5)	(7,6)	(8,6)	(9,7)	(10,8)	(11,8)	(12,9)	(13,9)	(14,10)
e	5	10	3	8	1	6	-1	4	9	2	7	0	5
$e_2 = 2e$	10	20	6	16	2	12	-2	8	18	4	14	0	—

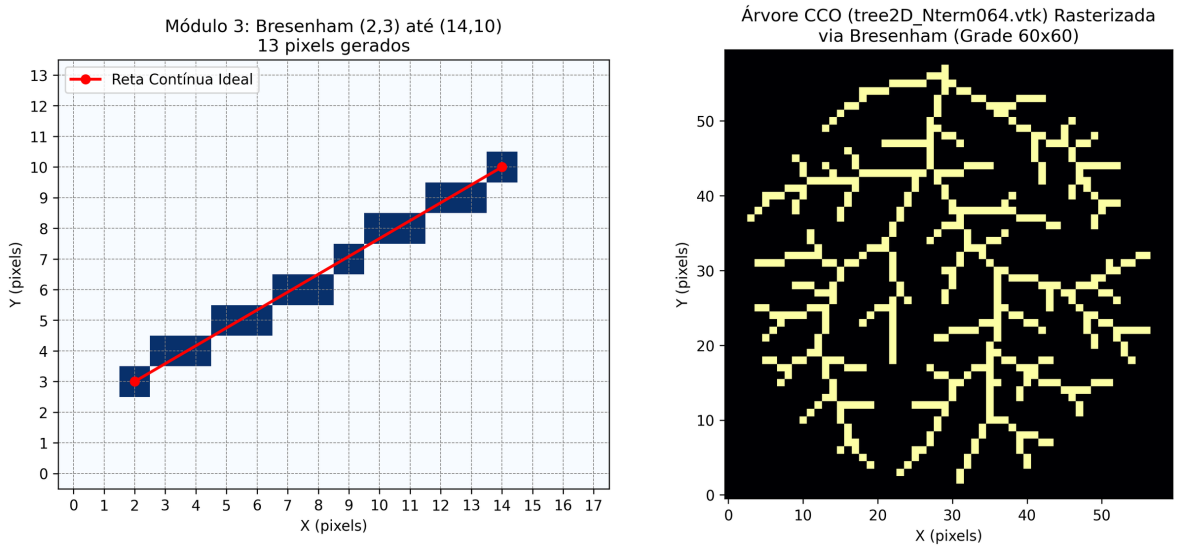


Figura 6: Módulo 3: pixels gerados pelo Bresenham para o exemplo da Tabela 3 (esquerda) e para a árvore CCO 2D projetada numa grade 60×60 (direita).

4.4 Módulo 4: rasterização de polígonos

O polígono de teste é côncavo, com seis vértices, e representa uma região de interesse. O Scanline com ET/AET preencheu 867 pixels nas linhas $y = 6$ a 42 (Figura 7). O painel do meio sobrepõe o contorno rasterizado por Bresenham, e o da direita compara as duas convenções de borda num quadrado 0..10. A convenção fechada (pixels sobre o contorno incluídos) gera $11 \times 11 = 121$ pixels. A semiaberta (esquerda e base incluídas, direita e topo excluídos) gera $10 \times 10 = 100$ pixels; é a convenção usada para que polígonos vizinhos não pintem duas vezes os pixels da aresta comum.

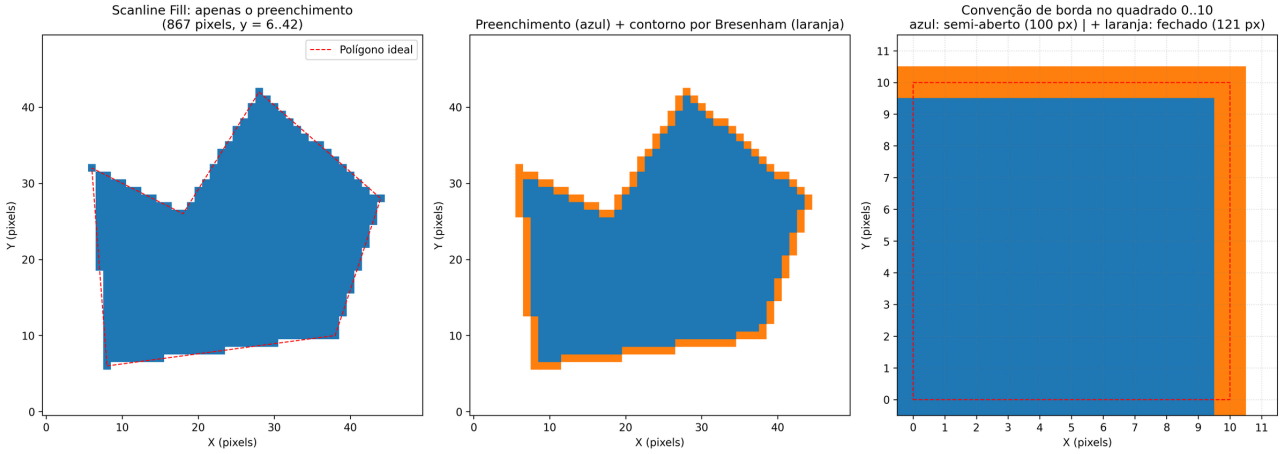


Figura 7: Módulo 4: Scanline Fill de um polígono côncavo e convenções de borda.

4.5 Módulo 5: aliasing e antialiasing

A Figura 8 e a Tabela 4 comparam a rasterização da mesma geometria com 1, 2×2 e 4×4 amostras por pixel. Com uma amostra, cada pixel só pode valer 0 ou 1: a reta vira uma escada e, na árvore, vasos com menos de um pixel de largura se fragmentam em pontos isolados, porque o centro do pixel ora cai dentro, ora fora do traço. Com $k \times k$ amostras há $k^2 + 1$ níveis de intensidade, e o erro em relação à referência cai cerca de 11 vezes para a reta e 8 vezes para a árvore com 4×4 amostras. Os vasos finos aparecem como traços contínuos e mais claros, proporcionais à área que cobrem.

Tabela 4: Níveis de intensidade e erro RMS em relação à referência 16×16.

Amostras por pixel	Níveis	Erro reta	Erro árvore	Custo relativo
1 (sem AA)	2	0,0797	0,1206	1×
2×2 (SSAA)	5	0,0229	0,0443	4×
4×4 (SSAA)	17	0,0073	0,0154	16×

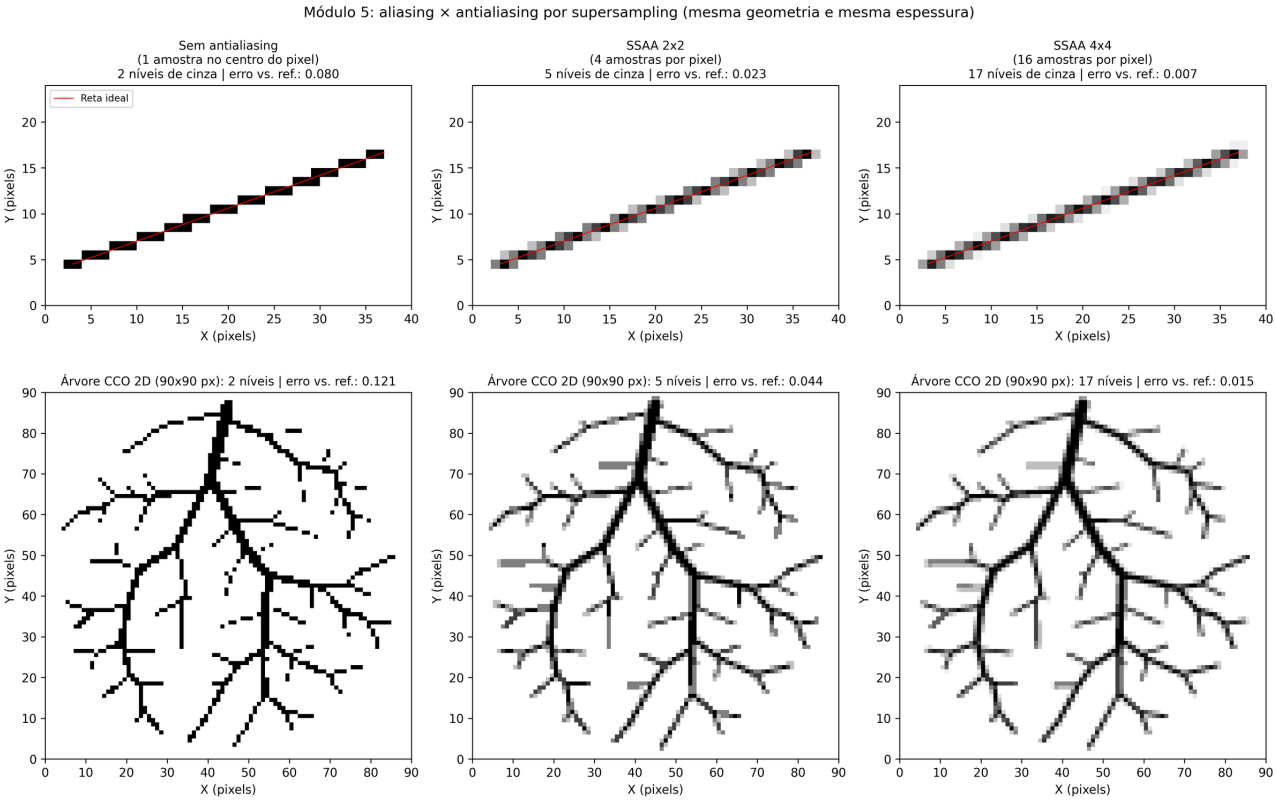


Figura 8: Módulo 5: supersampling implementado pelo grupo. Linha superior: segmento inclinado; linha inferior: árvore 2D a 90×90 pixels, com espessura proporcional ao raio.

No visualizador, a tecla `A` alterna entre sem antialiasing, MSAA e SSAA, fornecidos pelo VTK. A Figura 9 amplia uma região da imagem: sem antialiasing, as bordas dos tubos inclinados formam degraus; com MSAA e SSAA, os pixels de borda recebem cores intermediárias entre o tubo e o fundo.

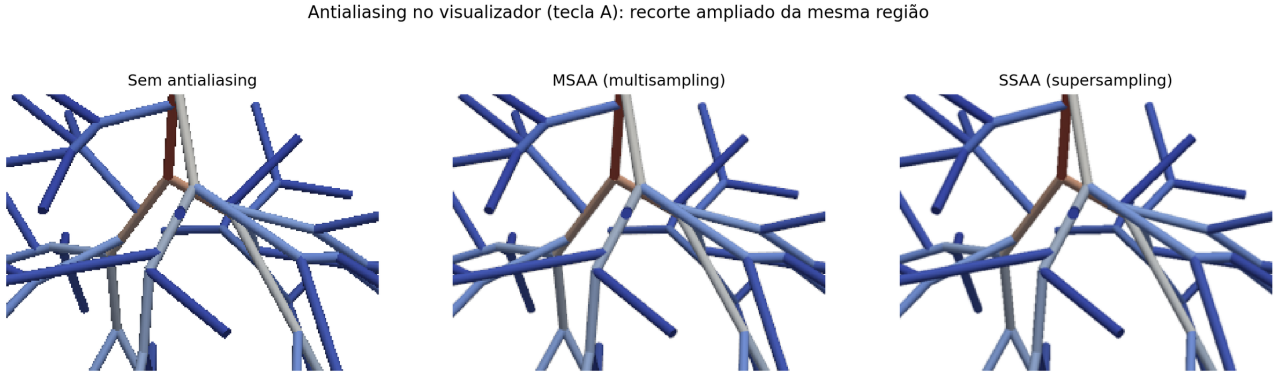


Figura 9: Módulo 5: antialiasing da API no visualizador 3D (mesma região ampliada).

4.6 Módulo 6: recorte de segmentos

Os 127 segmentos da árvore 2D foram recortados pela janela $[-0{,}022;\, 0{,}022] \times [-0{,}020;\, 0{,}025]$. A Figura 10 mostra a árvore antes do recorte e o resultado de cada algoritmo: os dois deixam 35 segmentos visíveis, e a comparação ponto a ponto não encontrou nenhuma divergência. Segmentos que apenas tocam a janela num ponto são descartados por ambos. A Figura 11 e a Tabela 5 comparam o custo.

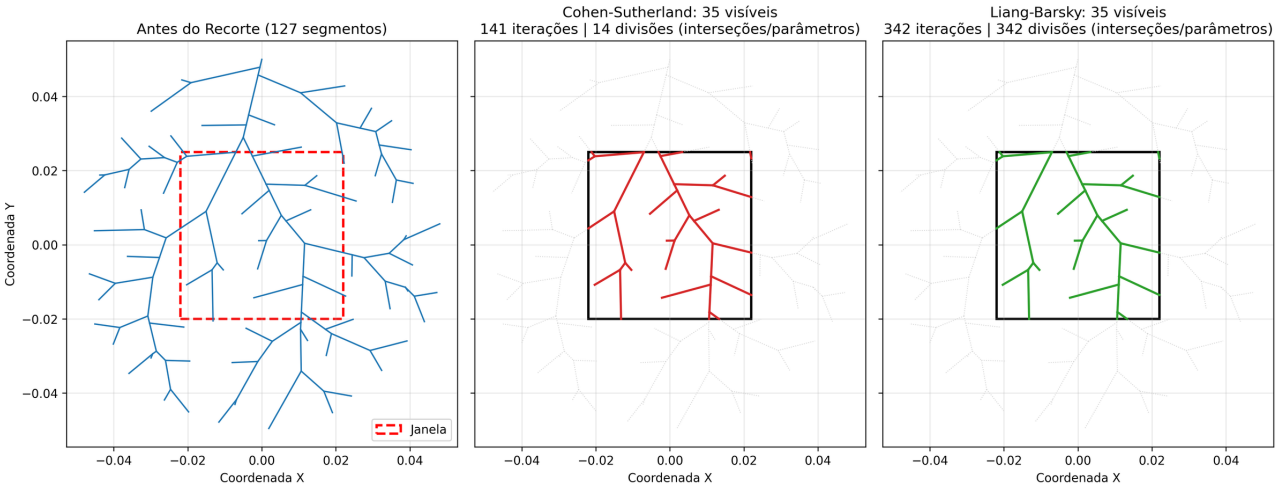


Figura 10: Módulo 6: árvore 2D antes do recorte e resultados de Cohen–Sutherland e Liang–Barsky.

Tabela 5: Custo do recorte dos 127 segmentos. "Iteração" é uma passada do laço (CS) ou um teste de borda (LB). O tempo é a média de 300 repetições em Python e depende da máquina.

Algoritmo	Visíveis	Iterações	Divisões	Tempo/segmento
Cohen–Sutherland	35	141	14	2,79 μs
Liang–Barsky	35	342	342	5,07 μs

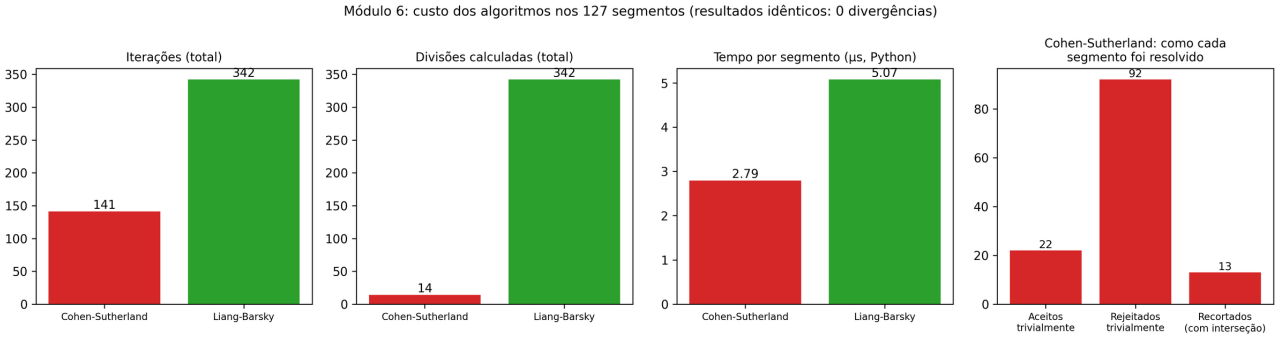


Figura 11: Módulo 6: custo dos dois algoritmos e classificação dos segmentos pelo Cohen–Sutherland (22 aceitos e 92 rejeitados trivialmente, 13 recortados).

4.7 Módulo 7: recorte de polígonos

Na Figura 12 (acima), um polígono de 7 vértices que ultrapassa os quatro lados da janela $[0, 10]^2$ foi recortado pelo Sutherland–Hodgman, resultando em um polígono de 8 vértices; a mesma janela em sentido horário é reorientada automaticamente e produz o mesmo resultado. Abaixo, um polígono côncavo em forma de "U" é recortado pela faixa $5 \leq y \leq 10$. O resultado correto são duas regiões

(os dois braços), mas o algoritmo devolve uma única lista de 8 vértices, ligando os braços por duas arestas sobrepostas de comprimento 4 ao longo de $y = 5$ (destacadas em vermelho). Se esse polígono fosse preenchido ou tivesse o contorno desenhado, apareceria um traço falso entre os braços. É o caso em que o Weiler–Atherton é o algoritmo adequado.

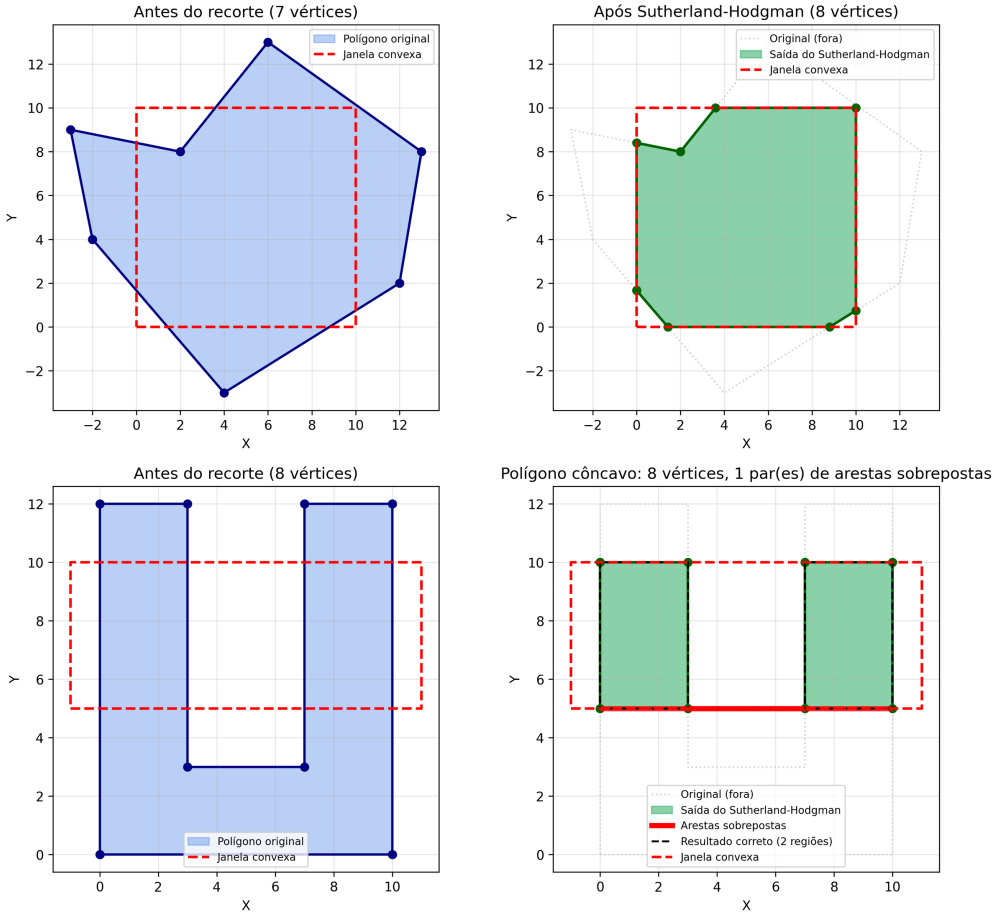


Figura 12: Módulo 7: Sutherland–Hodgman num polígono qualquer (acima) e num polígono côncavo cujo recorte correto tem duas regiões (abaixo).

5 Discussão

Comparação entre algoritmos de recorte. Na árvore CCO a maioria dos segmentos é curta em relação à janela, então quase todos ficam inteiramente dentro ou do mesmo lado de fora. O Cohen–Sutherland resolveu 114 dos 127 segmentos (90%) apenas com operações de bits, sem nenhuma divisão, e calculou só 14 interseções. O Liang–Barsky sempre executa os quatro testes de borda e calcula até quatro razões q_k/p_k por segmento, totalizando 342 divisões; nesta cena ele foi cerca de 1,8 vez mais lento (o tempo absoluto varia entre execuções e máquinas, mas o Cohen–Sutherland foi mais rápido em todas as nossas medições). A vantagem do Liang–Barsky aparece quando muitos segmentos cruzam a janela: o Cohen–Sutherland pode então calcular interseções intermediárias que serão descartadas (um extremo é movido para uma borda e ainda fica fora), enquanto o Liang–Barsky calcula cada extremo final uma única vez. O Cyrus–Beck, por sua vez, só se justifica quando a janela não é um retângulo alinhado aos eixos.

Decisões de implementação. (i) Um tubo por segmento, com pontos duplicados, para que o raio não seja interpolado entre vasos vizinhos. (ii) O pivô das transformações é o centroide atual, para que a árvore gire e escale "no lugar" e para que cada operação tenha inversa exata. (iii) Na comparação de antialiasing, a geometria e a espessura são idênticas nos três casos e só muda o número de amostras;

assim a diferença nas imagens vem apenas da amostragem. (iv) O Scanline usa intervalos semiabertos em y para tratar vértices compartilhados e arestas horizontais sem casos especiais; a versão fechada é a união de duas passadas, uma delas com o polígono espelhado em y. (v) O Sutherland–Hodgman valida que a janela é convexa e não degenerada e a reorienta para o sentido anti-horário, pois o teste de lado interno depende da orientação.

Limitações. O recorte e a rasterização didáticos operam sobre a projeção 2D (plano XY), e não dentro do *pipeline* 3D, onde o recorte é feito pela GPU contra o volume de visualização em coordenadas homogêneas. A malha de tubos é reconstruída a cada transformação, o que é aceitável para 127 segmentos, mas não escalaria para árvores com milhares de terminais; nesse caso a matriz deveria ser aplicada ao ator (`user_matrix`) ou na GPU. O Weiler–Atherton e o Cyrus–Beck foram apenas discutidos. As bifurcações não recebem esferas de junção, o que deixa pequenas descontinuidades entre tubos de raios muito diferentes.

6 Conclusão

Desenvolvemos uma aplicação que cobre os sete módulos do TP1 sobre os modelos CCO fornecidos: leitura própria do VTK, representação por tubos com cor e espessura proporcionais ao raio, transformações com matrizes homogêneas, quatro vistas e duas projeções, Bresenham, Scanline com ET/AET, supersampling, Cohen–Sutherland, Liang–Barsky e Sutherland–Hodgman. As partes implementadas pelo grupo e as fornecidas pelo PyVista/VTK estão separadas no código e na Tabela 1. Os experimentos quantificaram o ganho do supersampling, mostraram que os dois algoritmos de recorte de segmentos produzem o mesmo resultado com custos diferentes e evidenciaram a limitação do Sutherland–Hodgman com polígonos côncavos. Como trabalho futuro, no TP2, a mesma base receberá visibilidade (Z-buffer), modelos de iluminação, transparência e técnicas baseadas em raios, como a seleção de vasos por *ray casting*.

Agradecimentos

Os autores agradecem ao Departamento de Computação (DECOM) da Universidade Federal de Ouro Preto (UFOP) e ao professor da disciplina pelos modelos CCO disponibilizados.

Referências

[1] SCHREINER, W.; BUXBAUM, P. F. Computer-optimization of vascular trees. **IEEE Transactions on Biomedical Engineering**, v. 40, n. 5, p. 482–491, 1993.

[2] FOLEY, J. D.; VAN DAM, A.; FEINER, S. K.; HUGHES, J. F. **Computer Graphics: Principles and Practice**. 2. ed. Reading: Addison-Wesley, 1996.

[3] HEARN, D.; BAKER, M. P.; CARITHERS, W. **Computer Graphics with OpenGL**. 4. ed. Boston: Pearson, 2011.

[4] ANGEL, E.; SHREINER, D. **Interactive Computer Graphics: A Top-Down Approach with WebGL**. 7. ed. Boston: Pearson, 2015.

[5] BRESENHAM, J. E. Algorithm for computer control of a digital plotter. **IBM Systems Journal**, v. 4, n. 1, p. 25–30, 1965.

[6] LIANG, Y.-D.; BARSKY, B. A. A new concept and method for line clipping. **ACM Transactions on Graphics**, v. 3, n. 1, p. 1–22, 1984.

[7] CYRUS, M.; BECK, J. Generalized two- and three-dimensional clipping. **Computers & Graphics**, v. 3, n. 1, p. 23–28, 1978.

[8] SUTHERLAND, I. E.; HODGMAN, G. W. Reentrant polygon clipping. **Communications of the ACM**, v. 17, n. 1, p. 32–42, 1974.

[9] WEILER, K.; ATHERTON, P. Hidden surface removal using polygon area sorting. **ACM SIGGRAPH Computer Graphics**, v. 11, n. 2, p. 214–222, 1977.

[10] SCHROEDER, W.; MARTIN, K.; LORENSEN, B. **The Visualization Toolkit**. 4. ed. Kitware, 2006. Disponível em: https://vtk.org/.

[11] SULLIVAN, C. B.; KASZYNSKI, A. PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK). **Journal of Open Source Software**, v. 4, n. 37, p. 1450, 2019.